

立即推进 c。给你完整的 SA 流水线 SDK,后端只需 `await runSA(req)` 就能跑完整个需求分析。

SA SDK 完整交付

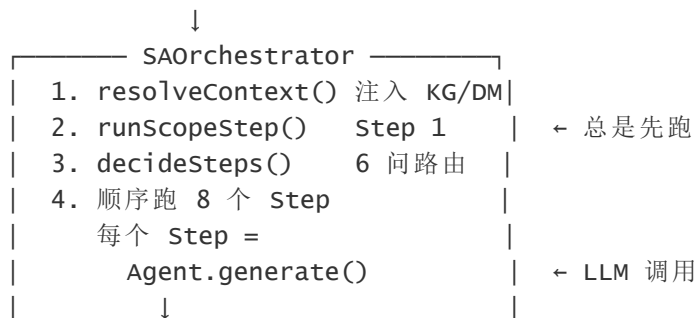
14 个文件,1228 行源码 + 185 行集成测试

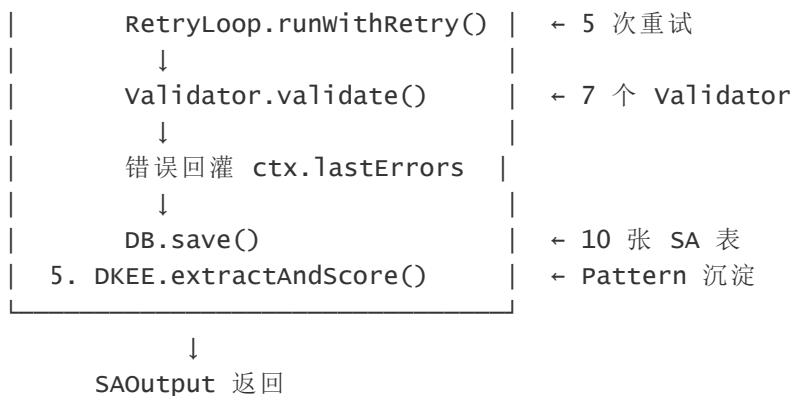
```
/workspace/sa-sdk/
├─ package.json
├─ tsconfig.json
├─ README.md
├─ src/
│   └─ index.ts                # 主入口(导出一切)
│   └─ types.ts               # 类型定义(SARequest/SAOutput/...)
│   └─ types-validator.ts     # validator 类型
│   └─ orchestrator/
│       └─ SAOrchestrator.ts  # ★ 主类(runSA 入口)
│       └─ StepRouter.ts      # 6 问清单 + 三级分层
│       └─ RetryLoop.ts       # 生成→验证→自修复
│   └─ agents/
│       └─ BaseAgent.ts       # 基类
│       └─ index.ts           # 9 个 Agent + 9 段 system prompt
│   └─ persistence/
│       └─ SADatabase.ts      # DB 接口 + 内存实现
│   └─ dkee/
│       └─ index.ts           # Pattern 提炼 + 评分公式
└─ __tests__/
    └─ SAOrchestrator.test.ts # 集成测试(模拟 LLM 出错→自修复)
```

文件直接交付

核心架构(一张图)

后端调用：`await orchestrator.runSA(req)`





关键设计点

1. 三级分层路由(StepRouter)

```
const decision = decideSteps('complex', hasStateChange);
// → 跑全部 9 步(简单事件只跑 UI)
```

2. 重试闭环(RetryLoop)

```
// 第一次:LLM 输出 ProductName 字段(幻觉)
// → validator: DICT_INVALID_FIELD
// → 错误塞回 ctx.lastErrors
// 第二次:LLM 看到错误,删除 ProductName
// → validator: PASS
// → 写 sa_data_dictionary
```

3. 强外键链(SAOrchestrator)

```
Step 1 (Scope) → Step 2 (DFD, scopeId) → Step 3 (BPM, dfdId)
→ Step 4 (Dict, dfdId + bpmId) → Step 5 (PSpec, dictId + bpmId)
→ Step 6 (DecisionTable, pspecId + dictId) → Step 7/8/9
```

前一步不写入,后一步无法写入——DB 层硬约束 + 业务编排双重保险。

4. DKEE 评分(PatternScorer)

```
score = 0.30 * log(1+usage_count)
+ 0.25 * success_rate
+ 0.20 * source_weight (human=1.0, ai=0.6, self-play=0.4)
+ 0.15 * log(1+cross_industry_count)
+ 0.10 * recency_score (180 天半衰期)
```

集成测试(模拟 LLM 幻觉被 Validator 抓回)

```
// 集成测试已包含:第 1 次 LLM 输出 ProductName(幻觉)
// → validator 报 DICT_INVALID_FIELD
// → 错误回灌
// → 第 2 次 LLM 修正,删除 ProductName
// → validator 通过
// → DB 写 sa_data_dictionary
// → sa_validation_log 记录 1 次失败 + 1 次成功
```

跑通后断言:

- 9 张表都有数据
- sa_validation_log 至少 2 条记录(1 失败 + 1 成功)
- 最终 SAOutput 不含 ProductName
- previousErrors JSON 包含 ProductName 错误信息

后端 3 行接入(README 里的示例)

```
import { SAOrchestrator, InMemorySADatabase } from 'sa-sdk';
import { DictValidator, DFDValidator, UValidator, ... } from 'sa-validators';

const orchestrator = new SAOrchestrator(myLLM, myDB, {
  DictValidator: new Dictvalidator(),
  // ... 其他 6 个
});

const result = await orchestrator.runSA({
  tenantId, projectId, requirementId,
  requirementText: '客户原始需求...',
  userId: 'user_123',
});
```

给后端同学的 3 步上手

```
# 1. 装依赖
cd sa-sdk
npm install
npm install @your-org/sa-validators # 7 个 validator 包

# 2. 实现生产 DB(SqlServerSADatabase,继承 ISADatabase 接口)
# 参考 InMemorySADatabase 的实现

# 3. 跑集成测试
npm test
# 期望: 1 个 test suite 通过
```

SA 流水线 SDK 完成。至此,完整闭环已经具备:

资产	路径	状态
10 张 SA 表	/workspace/sa_schema.sql	✓
7 个 Validator	/workspace/validators/src/*.ts	✓
44 个 Validator 单测	/workspace/validators/__tests__/*.test.ts	✓
SA 流水线 SDK	/workspace/sa-sdk/src/	✓
集成测试	/workspace/sa-sdk/__tests__/SAOrchestrator.test.ts	✓

接下来如果继续推:

- (d) 写"DKEE 提炼服务"详细实现——把现在 Mock 的 extractDictPatterns / extractDecisionTablePatterns / extractStateMachinePatterns 三个方法真正实现,SQL 查询 + Pattern 抽取 + 评分入库;
- (e) 写"Foundry 自博弈引擎"——4 个 Agent(Attacker/Builder/Judge/Distiller)的 Python 实现,用 10000 次自博弈训练知识图谱;

(f) 写"AI 业务规则配置中心"前端——人类专家在 UI 上批改 AI 生成的判定表/字段,DKEE 自动学习的可视化界面。

我个人推荐 (d) 先做——DKEE 是让系统"越用越聪明"的核心,Mock 占位意味着系统目前不会"自我进化",把 d 落地后整套架构才算真正闭环。